

Creating a Niagara N4 Module

Workbench *New Module* Wizard + IntelliJ IDEA project setup

niagara-research — operator workflow manual

Worked example: ColdRoomPan (cold-room control module)

Contents

1	Scope and prerequisites	1
2	Part A — Create the module in Workbench	2
2.1	Open the New Module wizard	2
2.2	Step 1 of 3 — module metadata and runtime profiles	2
2.3	Step 2 of 3 — dependencies	4
2.4	Step 3 of 3 — packages	5
2.5	Gotcha: you cannot go Back to Step 1	6
2.6	Finish — the generated project	7
3	Part B — Open the project in IntelliJ IDEA	7
3.1	Launch IntelliJ IDEA	7
3.2	Open the module folder	7
3.3	Trust the project	9
4	Part C — Configure the project	10
4.1	Set the SDK and language level	10
4.2	Edit <code>gradle.properties</code>	11
4.3	The two version-dependent edits	12
4.4	Result — environment ready	13
5	Part D — Programming: Gradle tasks and key files	13
5.1	Open the Gradle tool window (elephant icon)	13
5.2	The tasks that matter: build, clean, slotomatic	14
5.3	Key module files to keep in mind	15
5.4	Reference material	16
6	Part E — Building from WSL2 (command line)	16
6.1	Prerequisites in WSL2	16
6.2	Mandatory <code>-P</code> overrides	16
6.3	The build loop (from the project root)	17
6.4	Quick compile-only check	17
6.5	Tests	17
6.6	Operator-only activation steps	17
7	Summary	17

1 Scope and prerequisites

This manual documents, step by step, how an operator creates the empty **skeleton** of a Niagara N4 module with the Workbench *New Module* wizard, and then opens and configures that project in **IntelliJ IDEA** so it is ready to program. It is the “manual” path — the wizard generates a standard **gradle-niagara** project that you afterwards edit in the IDE.

The worked example throughout is the module used for the cold-room logic: **ColdRoomPan** (symbol **CRP**, vendor **Angeles**).

Prerequisites.

- Niagara Workbench (here: *Optimizer Supervisor* N4.14.0.162).
- IntelliJ IDEA (here: 2026.1.1).
- A full **JDK 8** (here: Azul *Zulu* 8) — Niagara builds require a real JDK, not just a JRE.
- Optionally Node.js, only if the module will build JavaScript (UX) parts.

Note. This produces exactly the kind of **gradle-niagara** project that the repository’s `docs/module-dev-workflow.md` runbook describes; once it is open and configured, you continue with the *edit* → *build* → *sign* → *deploy* → *test* loop.

2 Part A — Create the module in Workbench

2.1 Open the New Module wizard

In Workbench, open the **Tools** menu and choose **New Module** (Figure 1).

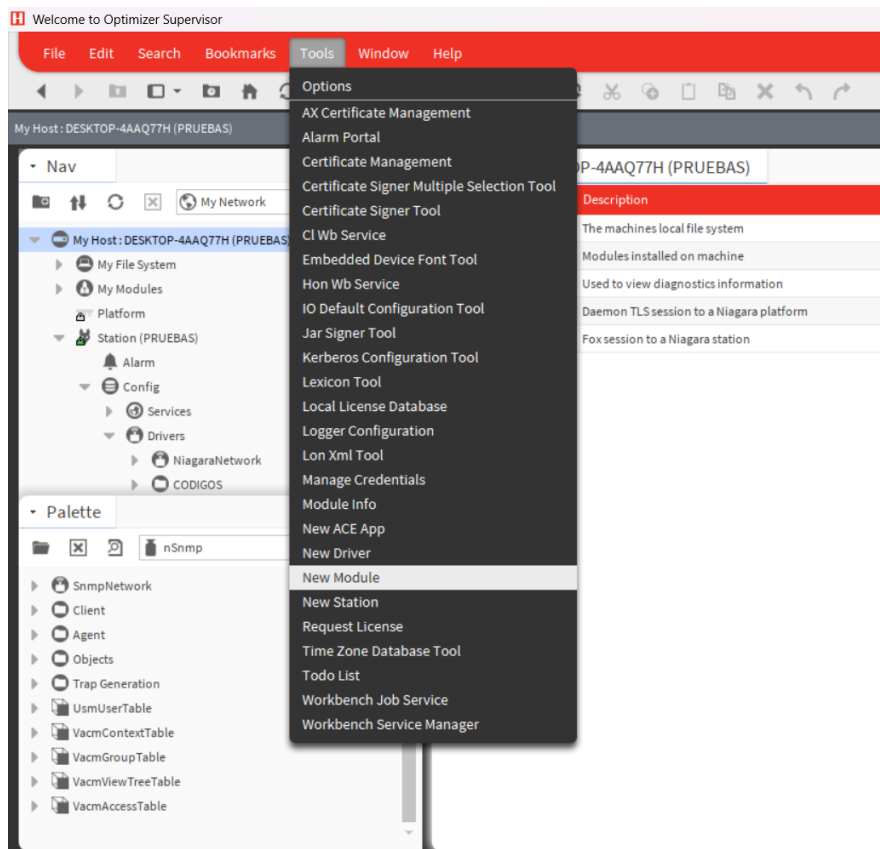


Figure 1: Workbench Tools → New Module.

2.2 Step 1 of 3 — module metadata and runtime profiles

The wizard opens on *Step 1 of 3*. The first field is the **Create Module under Directory** path — the folder where the project will be generated (Figure 2). In this example: `C:/modulos_niagara_n4/Cliente/Leon-Guanajuato/Paccadia`.

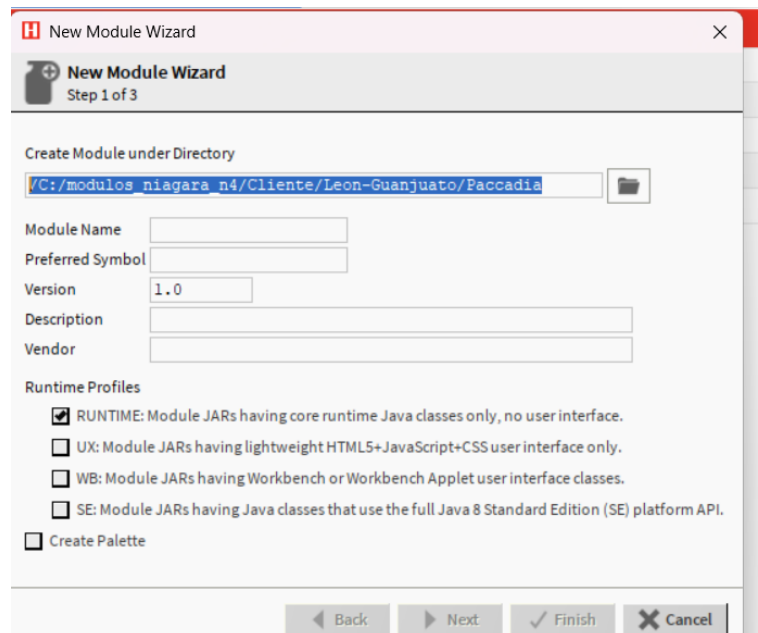


Figure 2: Step 1 of 3, empty. The directory path is selected first.

Fill in the remaining fields (Figure 3):

- **Module Name** — ColdRoomPan. The module’s technical name.
- **Preferred Symbol** — CRP. Short prefix/symbol for the module.
- **Version** — 1.0.
- **Description** — Logic Cold Room Panccadia.
- **Vendor** — Angeles.
- **Runtime Profiles** — which JAR profiles the module ships (see below).
- **Create Palette** — when checked, generates a palette file so the module’s components can be dropped from the Workbench palette.

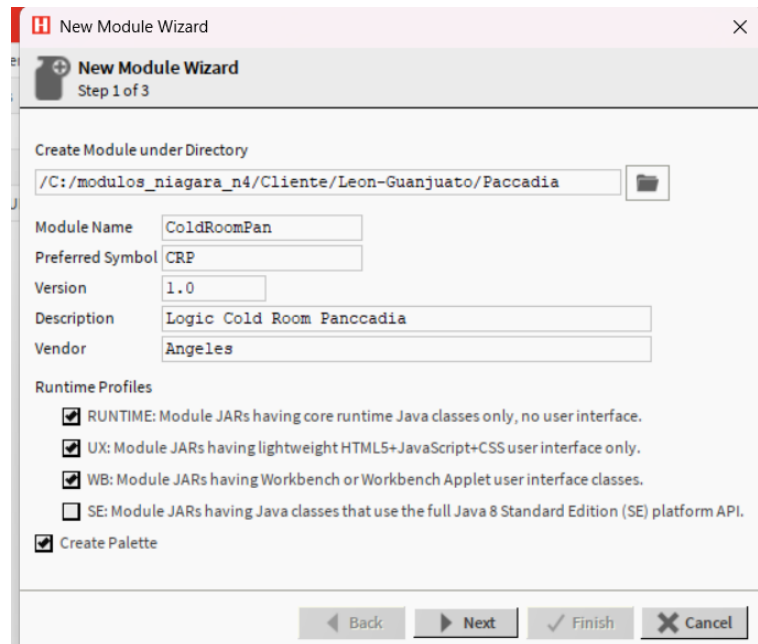


Figure 3: Step 1 filled in, with `RUNTIME` + `UX` + `WB` profiles and *Create Palette* checked (first attempt).

Runtime Profiles — what each one means.

`RUNTIME` (rt)

Core runtime Java classes only, no user interface.

UX Lightweight HTML5 + JavaScript + CSS user interface.

WB Workbench (or Workbench applet) user-interface classes.

SE Java classes that use the full Java 8 Standard Edition platform API.

Note. The recommended choice for this project is **pure `RUNTIME` (rt)**. The *UX* and *WB* profiles are only needed if you want a browser UI or a Workbench view inside the module — not required for control logic. Starting RT-only keeps the module small and simple; UX/WB can be added later if a Workbench interface is actually wanted.

2.3 Step 2 of 3 — dependencies

Clicking **Next** advances to *Step 2 of 3, Select Dependencies*. By default the module depends on `nre` and `baja` (both Tridium 4.14), shown in Figure 4. Use **Add** to pull in more modules (e.g. `control`, `kitControl`, `alarm`, `history`, `schedule`) from the picker in Figure 5.

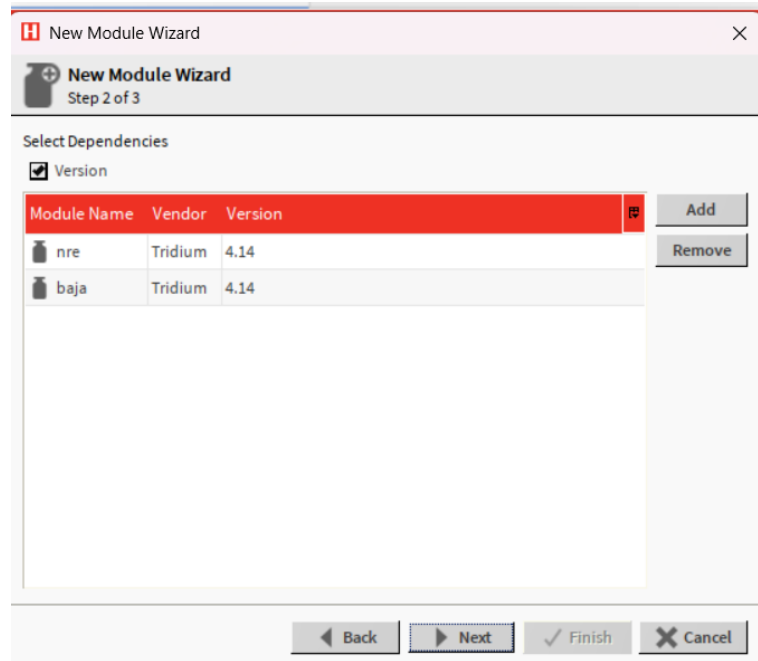


Figure 4: Step 2 of 3 — default dependencies `nre` and `baja`.

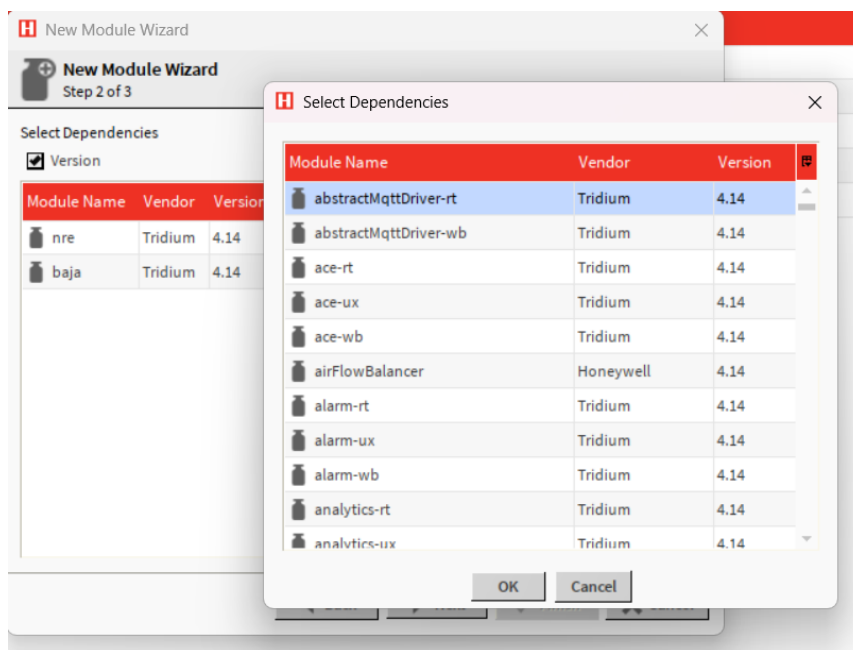


Figure 5: The *Select Dependencies* picker (Add).

Note. Dependencies can also be added *later*, directly in the module's build configuration, so it is fine to leave the defaults here and add what the logic needs during development.

2.4 Step 3 of 3 — packages

Next advances to *Step 3 of 3, Add Packages*. The wizard proposes one Java package per selected runtime profile (Figure 6): `com.angeles.ColdRoomPan` (rt), `com.angeles.ColdRoomPan.ui` (wb) and `com.angeles.ColdRoomPan.ux` (ux). The package prefix follows the vendor (`com.angeles`).

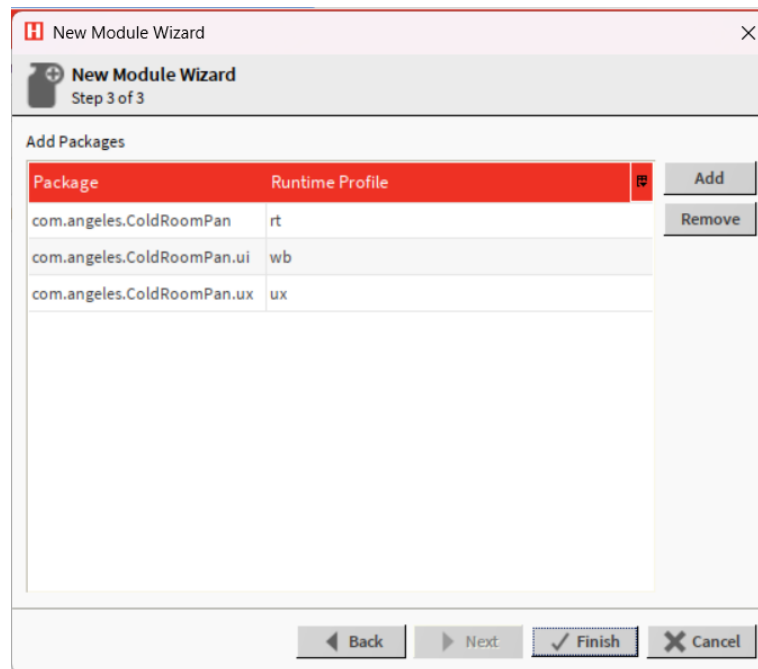


Figure 6: Step 3 of 3 — one package per profile (rt/wb/ux) in the first attempt.

2.5 Gotcha: you cannot go Back to Step 1

Once you press **Next** past Step 1, the **Back** button no longer returns to Step 1. To change the runtime profiles (here: to drop UX/WB and keep only RT) you must **cancel and start the wizard again**.

The second run keeps only **RUNTIME** plus *Create Palette* (Figure 7), which yields a single package `com.angeles.ColdRoomPan` with profile `rt` (Figure 8).

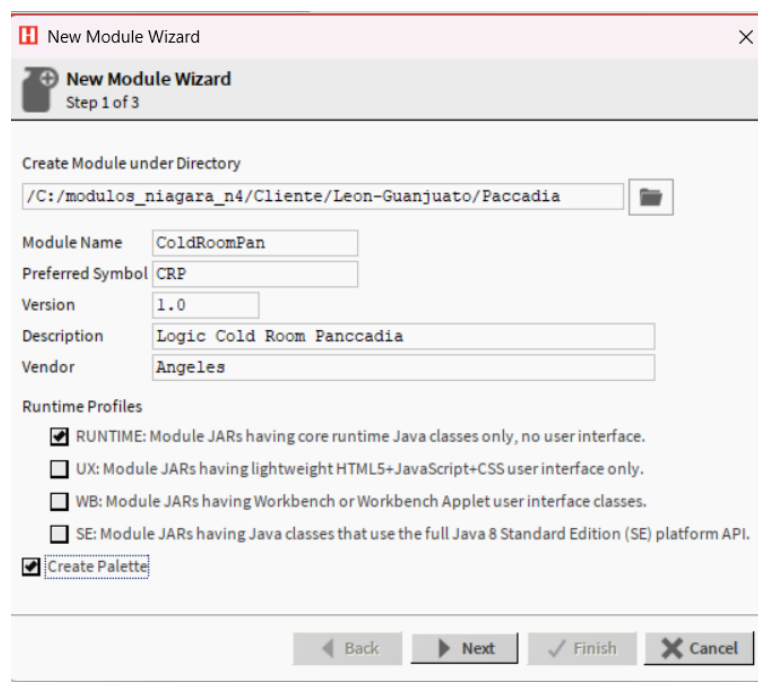


Figure 7: Second run — Step 1 with only RUNTIME + Create Palette.

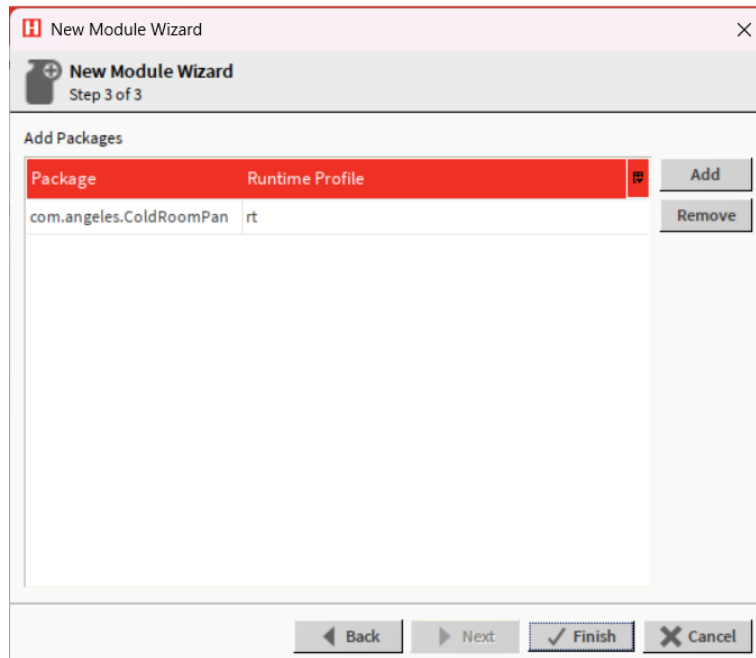


Figure 8: Second run — Step 3 with a single `rt` package.

2.6 Finish — the generated project

Pressing **Finish** creates the project. Browsing the target folder in Workbench shows the generated skeleton (Figure 9):

- `ColdRoomPan/` — the module source directory.
- `gradle/` — the Gradle wrapper support files.
- `build.gradle.kts`, `settings.gradle.kts` — Gradle build scripts.
- `gradle.properties` — build configuration (paths, JDK) — edited later.
- `gradlew`, `gradlew.bat` — the Gradle wrapper launchers.

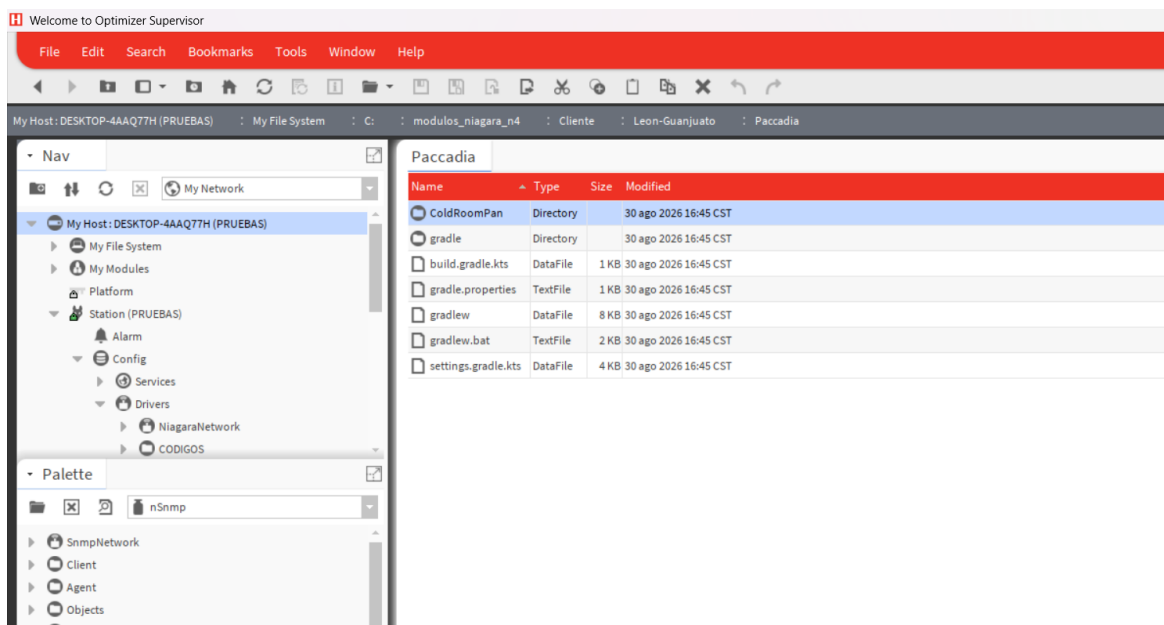


Figure 9: The generated project files in the target directory (Paccadia).

At this point the skeleton exists; what remains is to open it in the IDE and point the build at the correct Niagara install and JDK.

3 Part B — Open the project in IntelliJ IDEA

3.1 Launch IntelliJ IDEA

Open IntelliJ IDEA (Figure 10).

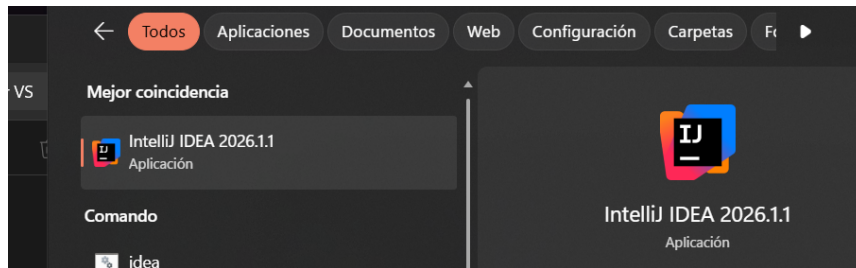


Figure 10: Launching IntelliJ IDEA 2026.1.1.

3.2 Open the module folder

On the Welcome screen (Figures 11 and 12) click **Open**, then browse to and select the module folder Paccadia (Figure 13) and press **Select Folder**.

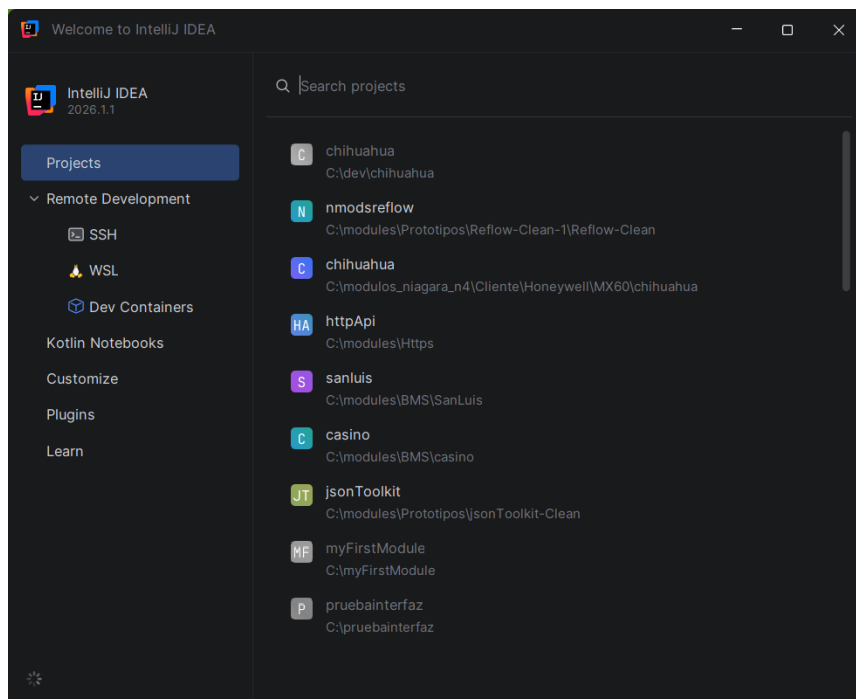


Figure 11: IntelliJ Welcome screen — recent projects.

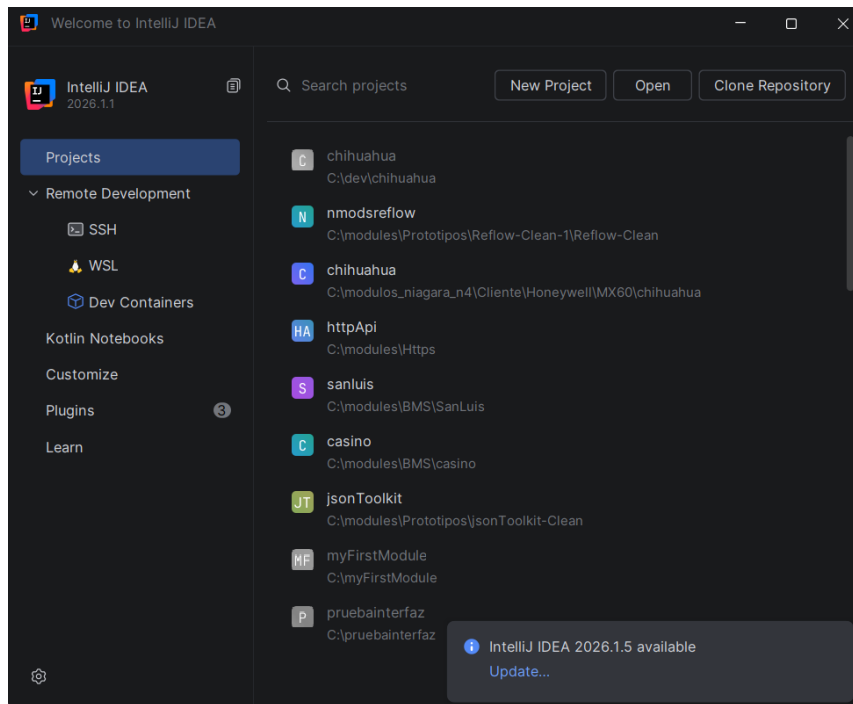


Figure 12: Welcome screen — *New Project / Open / Clone Repository*.

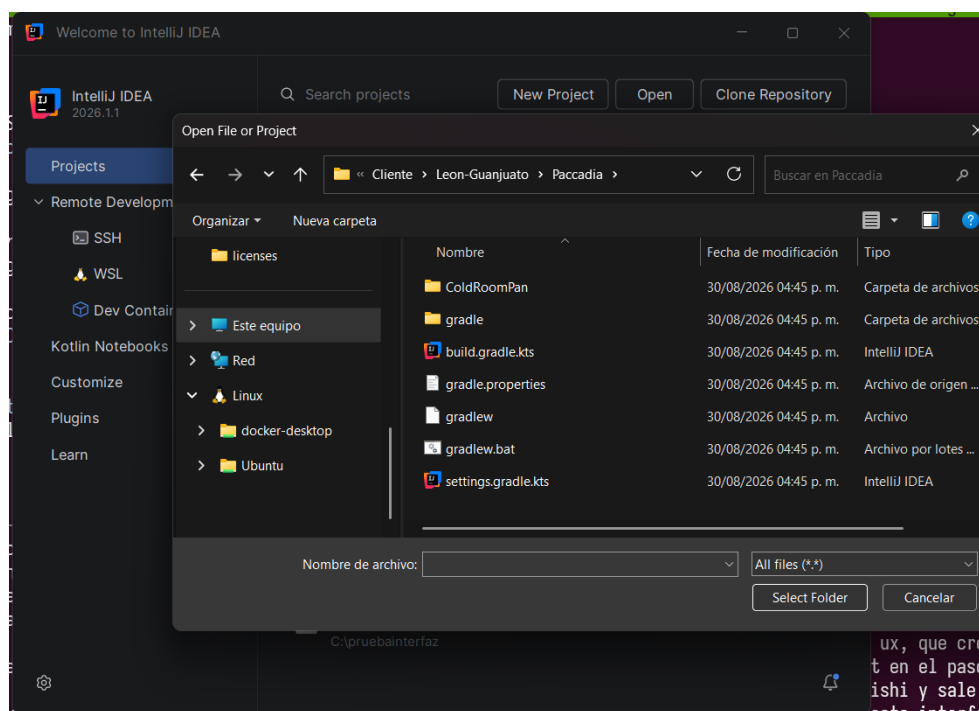


Figure 13: *Open File or Project* — select the Paccadia folder.

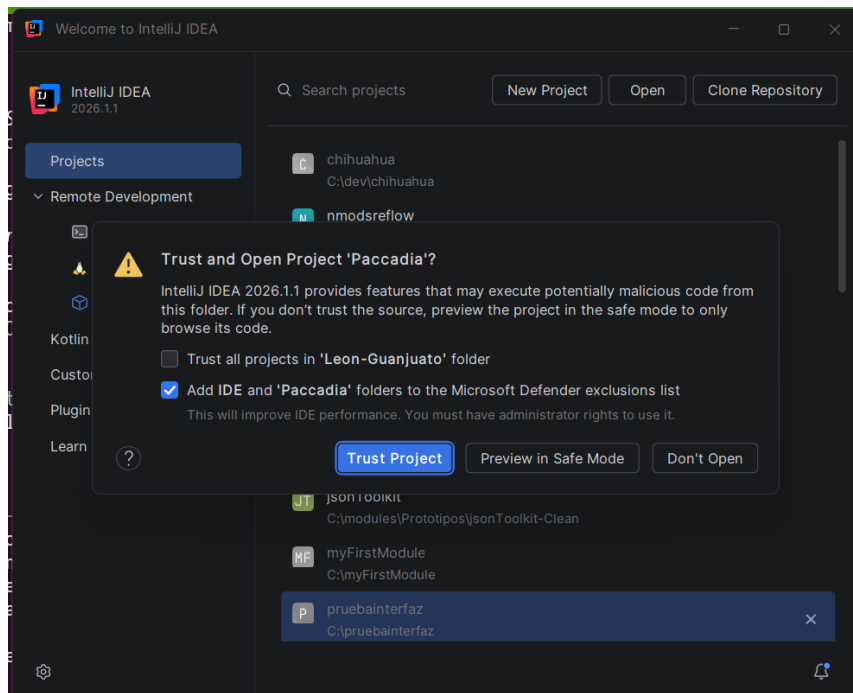


Figure 14: Clicking **Open** to browse for the project.

3.3 Trust the project

IntelliJ asks whether to trust the folder (Figure 15). Choose **Trust Project**. (The dialog optionally adds the IDE and project folders to the Microsoft Defender exclusion list to improve performance.)

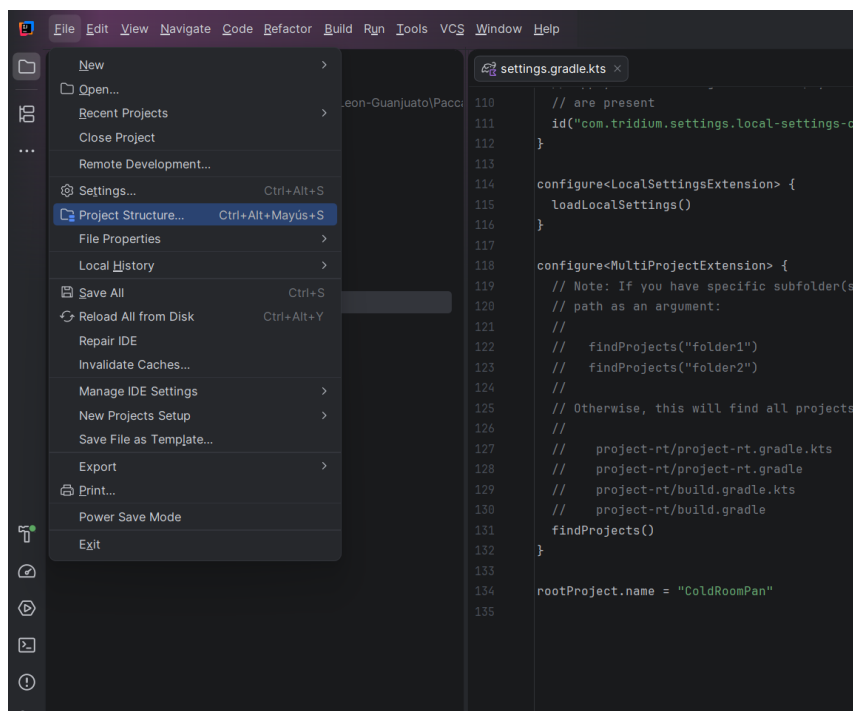


Figure 15: *Trust and Open Project 'Paccadia'?* $\rightarrow$ Trust Project.

After trusting, IntelliJ begins importing the Gradle project (Figure 16); wait for the import to finish.

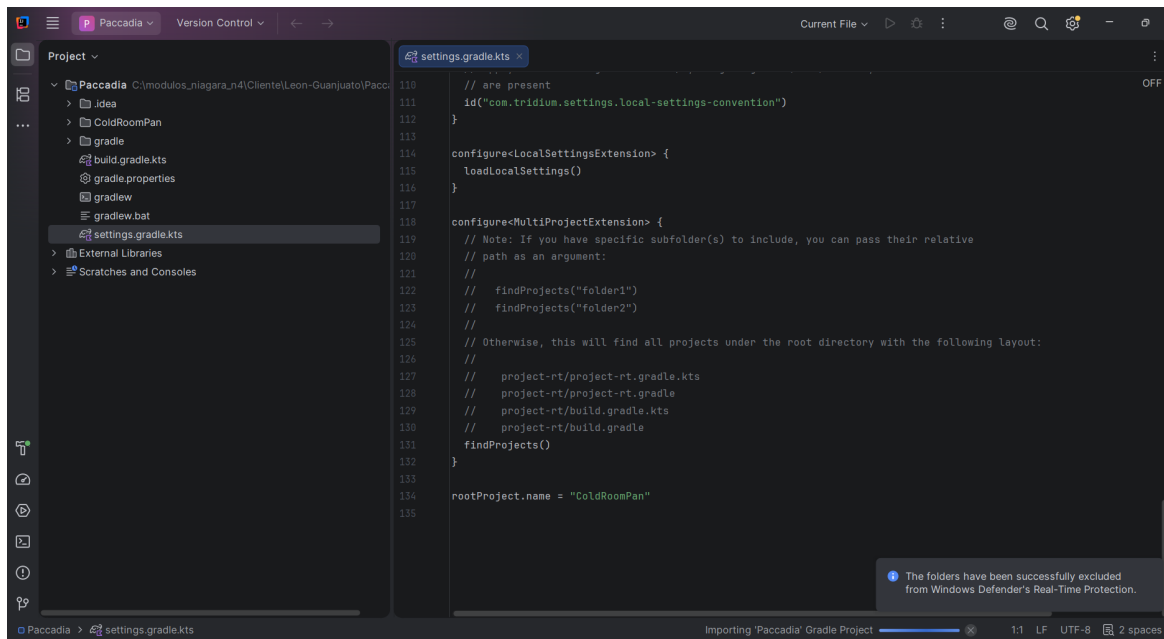


Figure 16: IntelliJ importing the Paccadia Gradle project.

4 Part C — Configure the project

4.1 Set the SDK and language level

Open **File** → **Project Structure** (Figure 17).

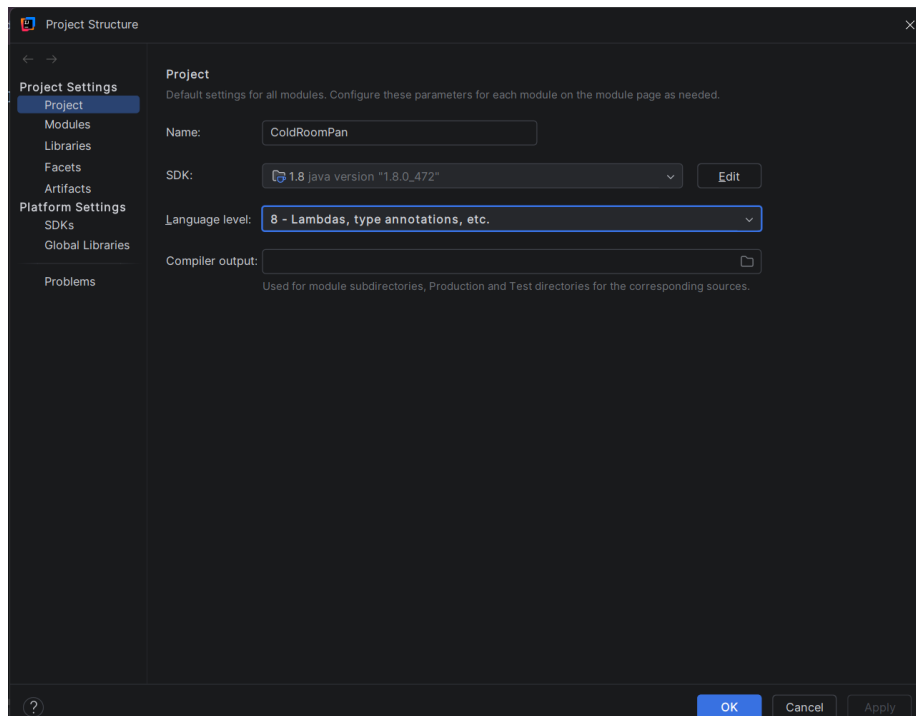


Figure 17: **File** → **Project Structure**.

Under **Project Settings** → **Project** (Figure 18) set the two values Niagara N4 requires:

- **SDK:** 1.8 (here 1.8.0_472).
- **Language level:** 8 - Lambdas, type annotations, etc.

Apply and wait a moment for the changes to take effect.

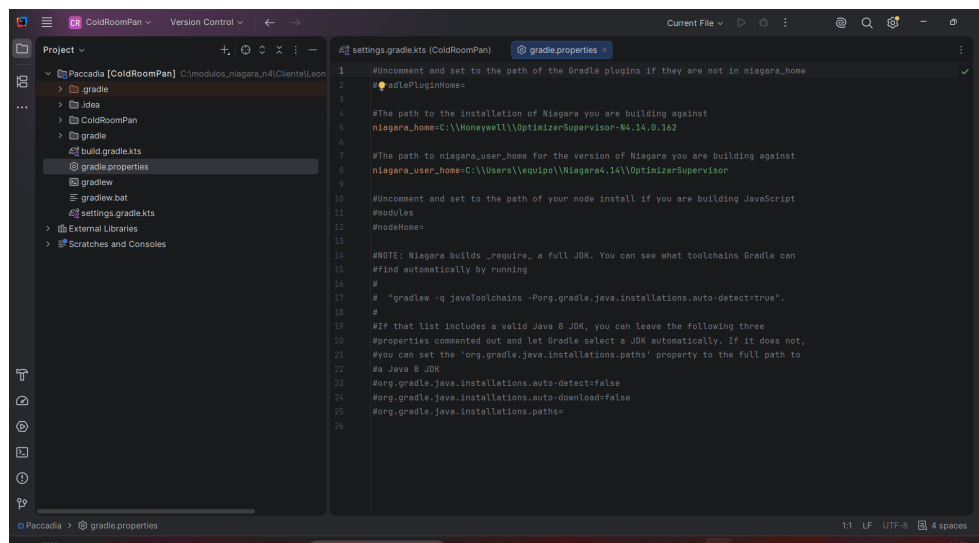


Figure 18: Project Structure — SDK 1.8 and language level 8.

4.2 Edit gradle.properties

Open gradle.properties (Figure 19). This file tells the build which Niagara install and which JDK to use.

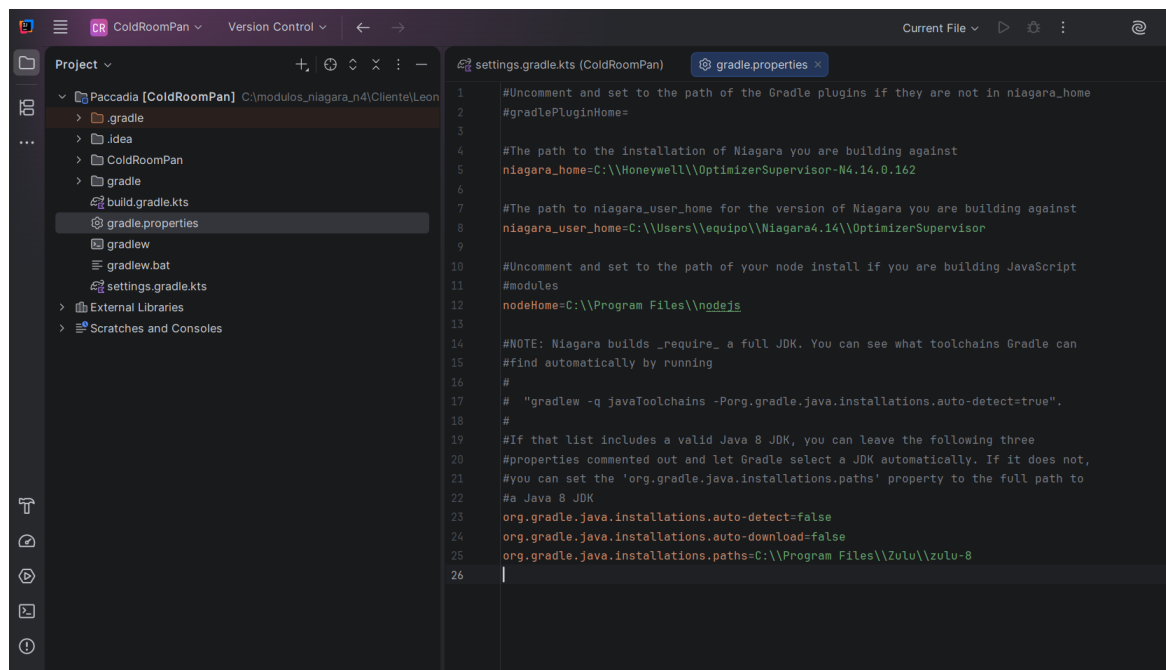


Figure 19: gradle.properties before editing (JDK lines commented out).

The relevant keys, as generated:

```
# The path to the installation of Niagara you are building against
niagara_home=C:\\Honeywell\\OptimizerSupervisor-N4.14.0.162

# The path to niagara_user_home for the version you are building against
niagara_user_home=C:\\Users\\equipo\\Niagara4.14\\OptimizerSupervisor
```

```
# Uncomment and set to the path of your node install if building JavaScript
#nodeHome=

#org.gradle.java.installations.auto-detect=false
#org.gradle.java.installations.auto-download=false
#org.gradle.java.installations.paths=
```

Apply the edits (uncomment and set the JDK toolchain, and Node if used):

```
niagara_home=C:\\Honeywell\\OptimizerSupervisor-N4.14.0.162
niagara_user_home=C:\\Users\\equipo\\Niagara4.14\\OptimizerSupervisor

nodeHome=C:\\Program Files\\nodejs

org.gradle.java.installations.auto-detect=false
org.gradle.java.installations.auto-download=false
org.gradle.java.installations.paths=C:\\Program Files\\Zulu\\zulu-8
```

Note. Pinning `org.gradle.java.installations.paths` to the Zulu 8 JDK and turning *auto-detect* / *auto-download* off guarantees the build uses the correct Java 8 toolchain instead of whatever Gradle might pick.

4.3 The two version-dependent edits

When you target a *different* Niagara version, two things change:

1. `gradle.properties` — update `niagara_home` (and `niagara_user_home`) to the path of that Niagara install.
2. `settings.gradle.kts` — update the Gradle version to the one that Niagara version builds with.

4.4 Result — environment ready

Once the Gradle project finishes importing with the correct SDK and paths, the Gradle tool window shows the project and its `ColdRoomPan-rt` sub-project with its tasks (Figure 20). The environment is now ready to start programming the module.

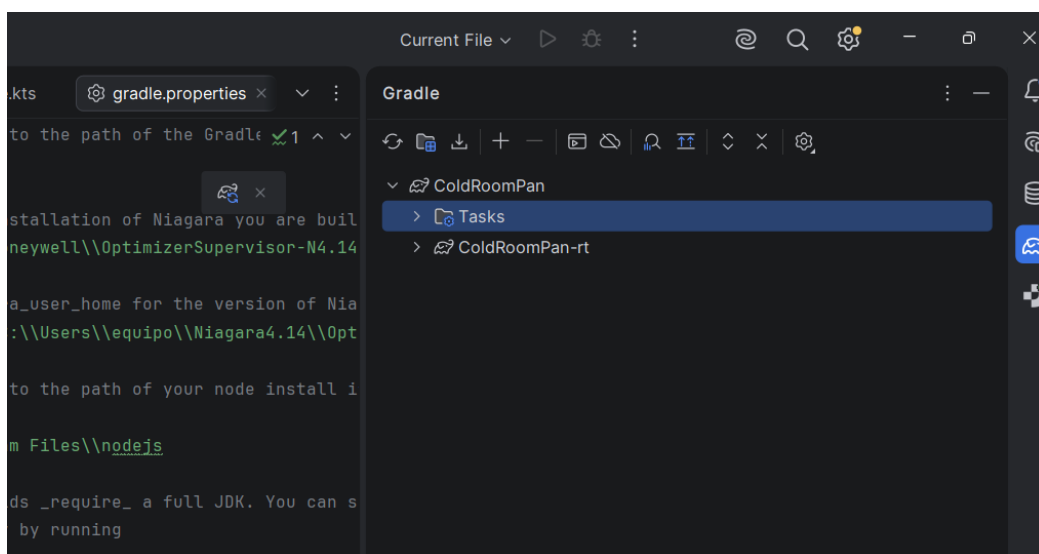


Figure 20: Gradle tool window — ColdRoomPan and ColdRoomPan-rt loaded.

5 Part D — Programming: Gradle tasks and key files

Once the project is open and configured you program the module and drive the build from the **Gradle** tool window. Remember that the **Gradle version depends on the Niagara version** (set in `settings.gradle.kts`).

5.1 Open the Gradle tool window (elephant icon)

Open the Gradle panel from the elephant icon on the right edge (Figures 21 and 22). It lists the root project `ColdRoomPan`, its **Tasks**, and the `ColdRoomPan-rt` sub-project.

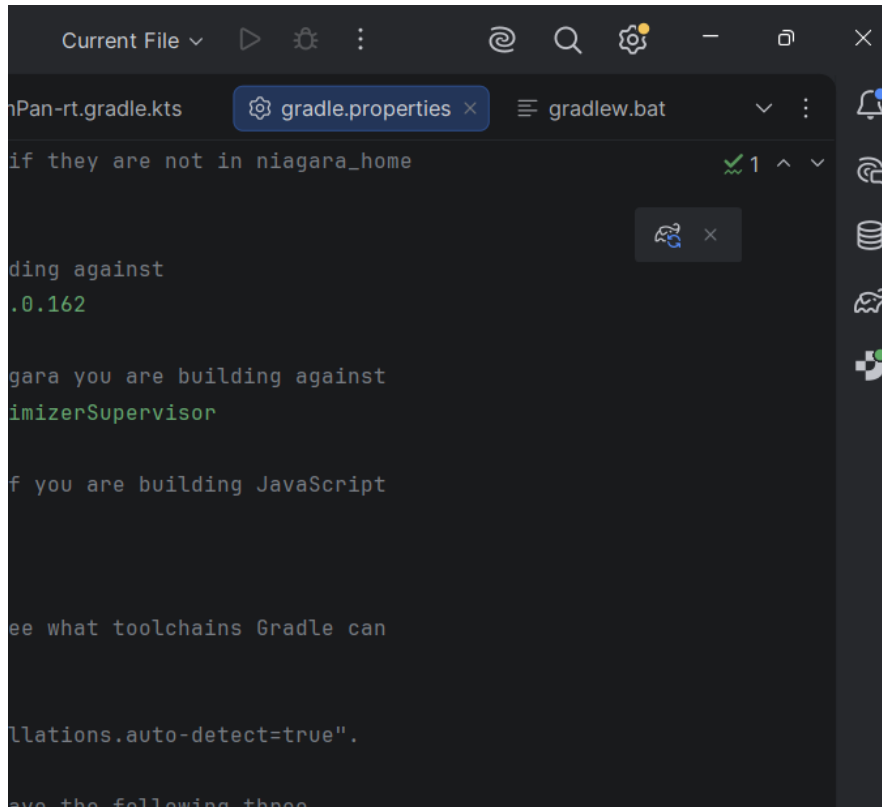


Figure 21: Opening the Gradle tool window.

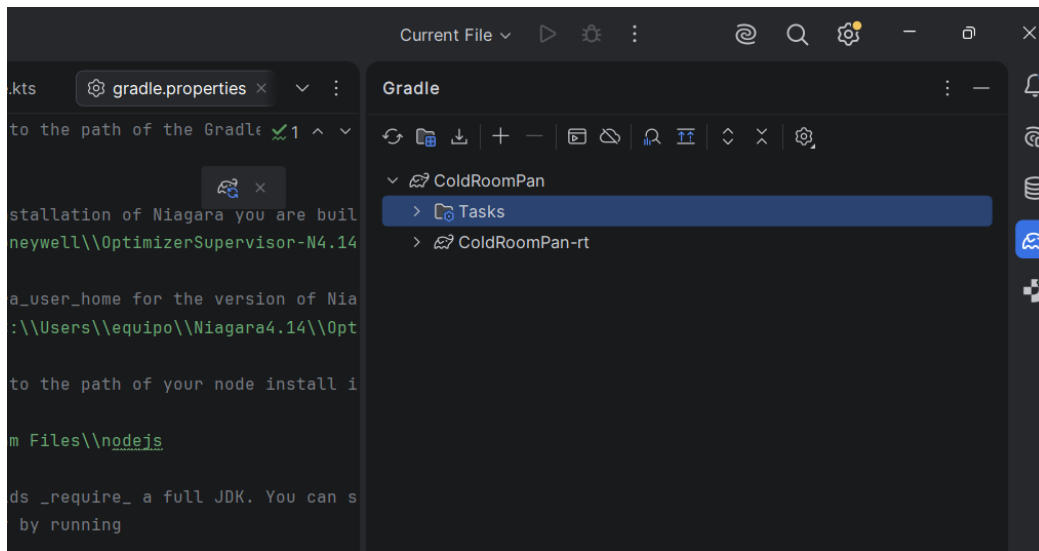


Figure 22: Gradle root project ColdRoomPan with its *Tasks* and the ColdRoomPan-rt part.

5.2 The tasks that matter: build, clean, slotomatic

Expand **Tasks** (Figure 23). Two groups matter:

- **build** — assemble, build, buildDependents, buildNeeded, classes, clean, jar, moduleTestClasses, moduleTestJar, testClasses. The two you use most are build and clean.
- **niagara** — migrateSlotomatic and slotomatic. slotomatic is the code-generation task.

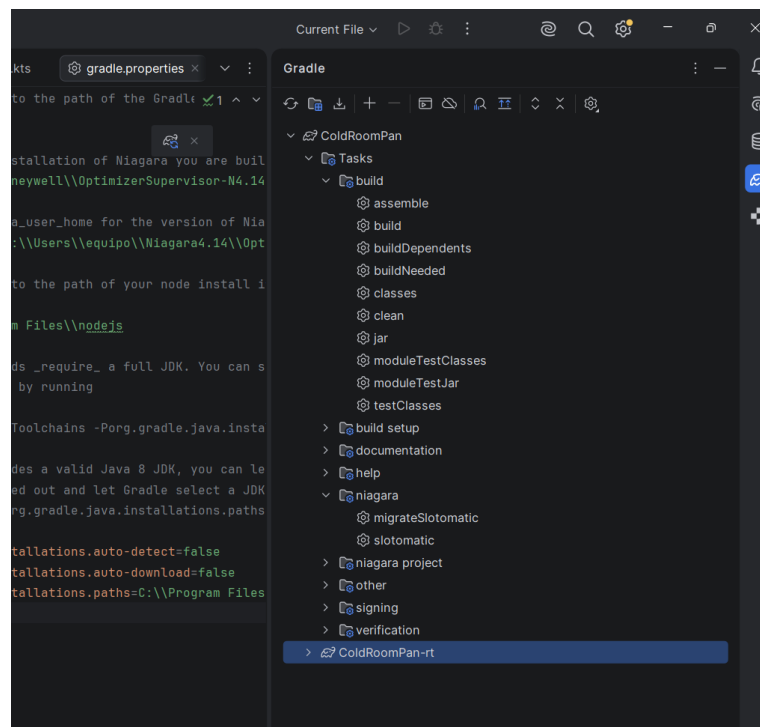


Figure 23: Tasks expanded: the *build* gear tasks and the *niagara* group with slotomatic.

Note. Slotomatic rule (documented in docs/module-dev-workflow.md §1.3/§3 and docs/module-best-practices.md §5.2): run `:slotomatic` *only* when a `@Niagara*` annotation changed. It writes the AUTO slot region of the class; **never** hand-edit inside the AUTO markers. A stale AUTO region causes a compile/runtime failure. Typical loop: `clean` → `slotomatic` (*only on annotation change*) → `build`.

5.3 Key module files to keep in mind

The ColdRoomPan-rt part contains the Niagara descriptor files (Figure 24):

module.palette

BOG of components for drag-and-drop from the Workbench palette. Ship it for any module that exports reusable components (ungated, lazy, fault-tolerant). Documented: `module-best-practices.md` [B634].

module-permissions.xml

A *Java Security Manager permission-request manifest*. It declares which restricted host capabilities the module's own *code* needs granted — network sockets, file access, native libraries, authentication, system time, backups, keystore — so the module can run protected operations without an `AccessControlException`. Root `<permissions>`, with `<niagara-permission-groups type="station|workbench|all">` blocks holding `<req-permission>` / `<opt-permission>` entries (`<name>`, `<purposeKey>`, optional `<parameters>` such as hosts/ports). It does **not** control how the module looks, and it is **not** per-user RBAC. The devkit emits it as a fully-commented stub (no extra permissions requested); the `chihuahua` module ships exactly that stub. Doc: `devguide security/requestingPermissions.txt` §1, §2.1.1, §2.2; build ref `build.txt:160`.

module.lexicon

Localization strings for the module.

niagara-module.xml / **module-include.xml**

The exported `<type>` list that Slotomatic reads.

ColdRoomPan-rt.gradle.kts

The part build script: `moduleManifest` (`moduleName`, `runtimeProfile=rt`), dependencies (`nre(":nre")`, `api(":baja")`, `moduleTestImplementation(":test-wb")`), and the `bajadoc` task (`includePackage("com.angeles.ColdRoomPan")`).

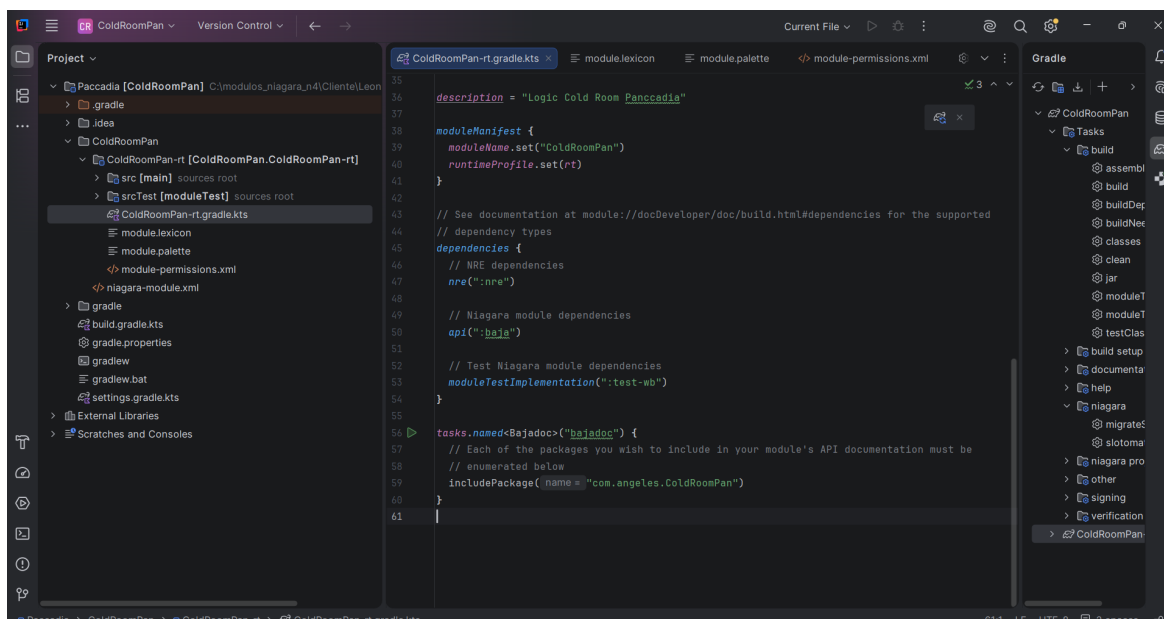


Figure 24: The ColdRoomPan-rt part: `module.palette`, `module-permissions.xml`, `module.lexicon`, `niagara-module.xml`, and `ColdRoomPan-rt.gradle.kts`.

5.4 Reference material

- Local tooling: `\\ws1.localhost\Ubuntu\home\cristian\modulos_niagara_n4\niagara-tools`
- Real worked example to study a full module's structure: `\\ws1.localhost\Ubuntu\home\cristian\modulos_niagara_n4\Cliente\Honeywell\MX60\chihuahua`
- Repository runbooks that document this part: `docs/module-dev-workflow.md` and `docs/module-best-pr`

6 Part E — Building from WSL2 (command line)

The project is created on Windows (the wizard) but can be developed and built from a WSL2 (Ubuntu) shell — the pattern used for the `chihuahua` module. This section documents that build loop. Source of truth: the `chihuahua` project’s `BUILD_WORKFLOW.md` / `CLAUDE.md`.

6.1 Prerequisites in WSL2

- A **JDK 8** installed in WSL: `sudo apt-get install -y openjdk-8-jdk` (lands at `/usr/lib/jvm/java-8-openjdk-amd64`).
- The Niagara install reachable from WSL at `/mnt/c/Honeywell/OptimizerSupervisor-N4.14.0.162`.

6.2 Mandatory `-P` overrides

`gradle.properties` points at Windows `C:\` paths that do not exist inside WSL, so every WSL build must override them on the command line:

```
-Pniagara_home=/mnt/c/Honeywell/OptimizerSupervisor-N4.14.0.162
-Porg.gradle.java.installations.paths=/usr/lib/jvm/java-8-openjdk-amd64
```

6.3 The build loop (from the project root)

```
# Clean + (regenerate AUTO region) + build the signed jar
./gradlew :ColdRoomPan-rt:clean :ColdRoomPan-rt:slotomatic :ColdRoomPan-rt:jar \
-Pniagara_home=/mnt/c/Honeywell/OptimizerSupervisor-N4.14.0.162 \
-Porg.gradle.java.installations.paths=/usr/lib/jvm/java-8-openjdk-amd64
```

- `slotomatic` runs in WSL and regenerates the AUTO GENERATED region from the `@Niagara*` annotations. Run it *only* when an annotation changed; otherwise `clean + jar` is enough. “Cannot transform ... no metadata” warnings for `srcTest/` are benign.
- The signed jar is produced by the `niagara-signing` plugin and lands in `$niagara_home/modules/ColdRoomPan-rt.jar`; copy it to the target station’s `modules` directory to deploy.
- **Version bump**: when you add or change a *frozen* slot (`@NiagaraProperty` / `@NiagaraAction`), bump `defaultModuleVersion` in the root `build.gradle.kts` (`chihuahua`’s `deploy.sh -bump`), or the new slot will not appear.

6.4 Quick compile-only check

Before a full build you can validate that the sources compile (useful while integrating code):

```
./gradlew :ColdRoomPan-rt:compileModuleTestJava \
-Pniagara_home=/mnt/c/Honeywell/OptimizerSupervisor-N4.14.0.162 \
-Porg.gradle.java.installations.paths=/usr/lib/jvm/java-8-openjdk-amd64
```

Note. Signing: the `niagara-signing` plugin signs the jar using the certificate store in `niagara_user_home`. If signing fails under WSL, also override it: `-Pniagara_user_home=/mnt/c/Users/equipo/Niagara4.14/OptimizerSupervisor`. Deploying a custom module to a locked-down OEM (Honeywell) install requires a certificate the station trusts (here: the **ANGELES** identity).

6.5 Tests

Do **not** gate the build on `gradle test` / `niagaraTest`: the `com.tridium.niagara-module 7.6.17` plugin has a known bug (`moduleTestAnnotationProcessor` produces no metadata, so `@NiagaraType` tests are silently skipped, `Total tests run: 0`). Test files in `srcTest/` are documentation-only skeletons.

6.6 Operator-only activation steps

The build/deploy runs from WSL, but these are done by the operator on the station:

- **Restart the station** — for Java changes and frozen slots.
- **Close and reopen Workbench** — if a frozen slot/action still does not appear after the station restart (Workbench's client type cache does not refresh on a station restart).
- **Smoke test** — verify on the live station.

Note. Gotcha: `compileModuleTestJava` pollutes `module-include.xml` with test types — `git checkout` that file before committing.

7 Summary

1. Workbench: **Tools** → **New Module**.
2. Step 1: path + Module Name, Preferred Symbol, Version, Description, Vendor; choose **RUNTIME** only; check **Create Palette**. (Remember: no going Back to Step 1.)
3. Step 2: dependencies (defaults `nre`, `baja`; add more here or later).
4. Step 3: one `rt` package `com.<vendor>.<Module>`; Finish.
5. IntelliJ: **Open** the generated folder → **Trust Project** → wait for the Gradle import.
6. **Project Structure**: SDK 1.8, language level 8.
7. `gradle.properties`: set `niagara_home`, `niagara_user_home`, `nodeHome` (if needed), and pin the Zulu 8 JDK toolchain.
8. Per Niagara version: adjust `niagara_home` in `gradle.properties` and the Gradle version in `settings.gradle.kts`.

Example values used in this manual.

Field	Value
Module Name	ColdRoomPan
Preferred Symbol	CRP
Version	1.0
Description	Logic Cold Room Panccadia
Vendor	Angeles
Package (rt)	<code>com.angeles.ColdRoomPan</code>
Directory	<code>C:/modulos_niagara_n4/Cliente/Leon-Guanajuato/Paccadia</code>
Niagara home	<code>C:/Honeywell/OptimizerSupervisor-N4.14.0.162</code>
JDK	Zulu 8 (<code>C:/ProgramFiles/Zulu/zulu-8</code>)